



INDIVIDUAL ASSIGNMENT COVERSHEET

Family Name: XXXX

Given Name: XXXXX

Student Number: 121XXXXX

Lecturer's Name: Ali Noori

Subject Name: ICT201

Assignment Title: Assessment 4 (Individual)

Declaration

(This declaration must be completed by the student, or the assignment will not be marked.)

I certify the following:

- I have read and understood the *Student Academic Misconduct Policy*
- This assignment is my own work based on my personal study and or research.
- I have acknowledged all material and sources used in the preparation of this assignment including any material generated in the course of my employment.
- The assignment has not previously been submitted for assessment.
- I have not copied in part or in whole or otherwise plagiarised the work of other students.
- I have read and I understand the criteria used for assessment.
- The assignment is within the word and page limits specified in the unit outline.
- The use of any material in this assignment does not infringe the intellectual property / copyright of a third party.
- I understand that this assignment may undergo electronic detection for plagiarism, and an anonymous copy of the assignment may be retained on the database and used to make comparisons with other assignments in future.
- By completing this coversheet in full and submitting this assignment electronically, I am bound by the conditions of the KOI's *Student Academic Misconduct Policy* and the declaration on this coversheet.

Signature

31 / 08 / 2025
Date

Table of Contents

1. Introduction	3
1.1 Firmware and Secure Boot	3
2. UEFI vs BIOS: Architectural Differences	3
3. How Secure Boot Works	4
3.1 Initialization of UEFI Firmware	4
3.2 Verification of Firmware Integrity (Certificate Authorities)	4
3.3 Digital Signatures	5
3.4 Loading the Bootloader- (Bootloader are verified)	5
3.5 Operating System Verification	5
4. Role of TPM (Trusted Platform Module)	5
4.1 Storing Encryption Keys	5
4.2 Platform Integrity Verification	5
4.3 Binding and Sealing of Data	5
4.4 Support for Advanced Security Features	6
5. Secure Startup in Operating Systems	6
5.1 Windows Secure Startup	6
5.2 Linux Secure Startup	6
6. Real-World Use Case	7
6.1 Secure Boot	7
6.2 TPM: Hardware-Based Key Protection	7
6.3 BitLocker: Full Disk Encryption	7
6.4 End-to-End Chain of Trust	8
7. Challenges and Limitations	8
7.1 Dual-Boot Restrictions	8
7.2 Firmware Updates	8
7.3 Secure Boot Bypass Methods	8
7.4 Usability vs. Security Trade-off	8
8. References	9

Secure Startup: Understanding Firmware, Secure Boot, and TPM in Modern Operating Systems

1. Introduction to Firmware and Secure Boot

Firmware is a fundamental software specific to a computer hardware unit, and it is stored directly on the hardware. It is the first to be activated before the operating system which controls the initial steps of booting the computer as well as, on a general level, firmware gets the hardware ready and makes an environment for the operating system to run smoothly.

Upon computer start, firmware firstly checks the processor, memory, storage, and other devices to ascertain their proper functioning. Next, it chooses a bootloader, which is the one in charge of loading the operating system. This stage is very crucial because, if at any point, a harmful code is run at the very start, the whole system is jeopardized.

Consequently, the concept of Secure Boot was created. It is a UEFI firmware security feature that allows only trusted and verified software to be executed during the computer startup. Secure Boot verifies the digital signature of each bootloader, driver, and system file. Unauthorized or altered codes if any, will be stopped from running.

In brief, firmware is the one in charge of the startup process, and Secure Boot is the guard of that process allowing a secure and trustworthy environment from the first second the computer is turned on.

2. UEFI vs BIOS: Architectural Differences

Feature	Legacy BIOS	UEFI	Improvement (UEFI Advantage)
The Age of Technology	Used since the 1980s	Modern standard (post-2000)	Replaces outdated limitations with stable modern design
Operation Mode	Limited to 16-bit real mode	Supports 32-bit and 64-bit execution	Better performance and larger memory access
Memory Limitation	Maximum 1 MB addressable space	Supports >2 TB drives and large memory capacity	Enables large disk and modern hardware support
Interface	Simple text-based interface	Graphical interface with mouse support	User-friendly and easier configuration
Security	No signature verification, vulnerable to rootkits	Secure Boot supported with digital signature checks	Prevents execution of unauthorized/malicious code
Extensibility	Static code, adding new drivers/features is difficult	Modular architecture; drivers/apps can be added	Flexible for future hardware and software support
Compatibility	Works only with older operating systems	Supports modern OS with optional Legacy Boot	Works with both old and new systems

BIOS is a legacy 16-bit firmware with limited memory support and weak security. **UEFI** is a modern, extensible firmware supporting larger storage, Secure Boot, and enhanced security, performance, and flexibility.

3. How Secure Boot Works

As shown in the Flowchart Below Firmware initialization is the first process when the system is powered on. Now Secure Boot verification verifies the digital signatures of each boot component against the public keys of the embedded system which is provided by the vendors while manufacturing. If the keys are valid then it moves to the next step i.e. embedded public keys (Certificate authorities) and if it is not valid it again goes to the firmware initialization. The embedded public keys are provided by the vendor or hardware manufacturer and all the sets of keys are stored in the firmware. When one component's signature is valid it adds the next component in the chain for verification. When all the boot components are verified and valid the firmware loads the operating system kernel into memory. In the secure boot policy, endorsement-enabled and disabled options are included. Protected against malware protects our system from malicious or unauthorized software whose signature is missing or invalid.

Boot Sequence

The Secure Boot process follows a specific step-by-step structure that is executed every time the computer is started. At each step, the system ensures that only approved and verified software is loaded, thereby maintaining the integrity of the system.

Step 1: Initialization of UEFI Firmware

You must enable the UEFI feature in the BIOS settings to kickstart the process. The boot sequence begins with the UEFI firmware activation before performing a hardware check. The process will boot the system if everything checks out after the hardware check.

Step 2: Verification of Firmware Integrity (Certificate Authorities)

In this step, the UEFI firmware verifies its own integrity. This is done using the Platform Key (PK), which establishes a Root of Trust for the entire boot process. The keys, supplied

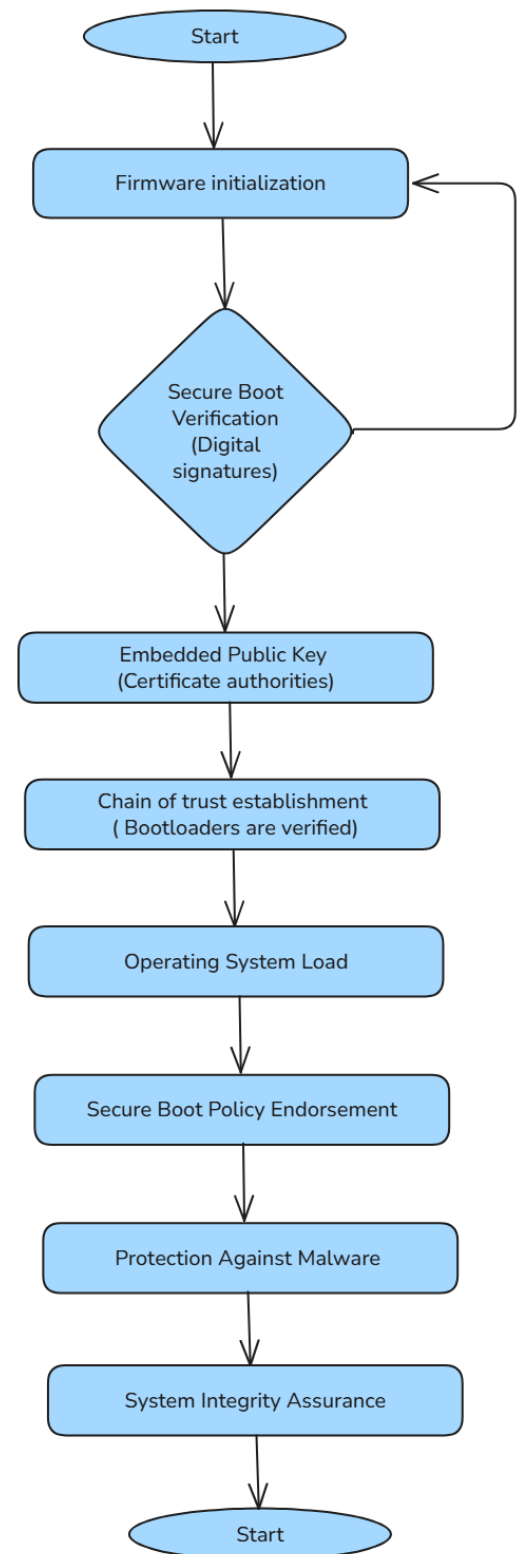


Figure-1: Flowchart of How Secure Boot Works

by the hardware vendor or manufacturer and approved by Certificate Authorities (CA), are stored in the firmware. These keys are used to determine which software components are trusted.

Step 3: Digital Signatures

The boot sequence will then check the digital signature of the bootloader and executable files against a database of trusted signatures. Signature checking is critical because it only allows files to execute if their signature checks out.

Step 4: Loading the Bootloader- (Bootloader are verified)

This is the stage where the UEFI loads the bootloader into the PC's memory. It is also the stage where initialization of the operating system kernel and passing control to it happens.

Step 5: Operating System Verification

The bootloader will further verify the integrity of the operating system kernel and any other essential components before loading them. Bootloaders will prevent the operating systems from loading if there are any unauthorized changes or malware.

4. Role of TPM (Trusted Platform Module)

Trusted Platform Module (TPM) is a special hardware chip that is connected to the motherboard and performs cryptographic operations to ensure the security of the computer. TPM plays an important role in operating system level security in addition to Secure Boot. Its main functions are as follows:

I. Storing Encryption Keys

The TPM securely stores cryptographic keys, which are many times more secure than regular memory. For example, the BitLocker feature in Windows uses keys stored in the TPM to perform hard disk encryption. As a result, even if the hard drive is connected to another computer, the data cannot be accessed.

II. Platform Integrity Verification

The TPM stores measurements at every step during the system boot process. The hash value of each component, from BIOS/UEFI to the bootloader, is recorded in the TPM. These values are then compared to the expected values. If any unauthorized changes occur, the TPM detects it and alerts Secure Boot.

III. Binding and Sealing of Data

TPM can lock data with specific hardware and software configurations. This is called sealing. That is, encrypted data can never be decrypted without the right hardware or the right boot environment. This protects the system from rootkits or firmware-level malware.

IV. Support for Advanced Security Features

Using TPM not only secures Secure Boot, but also other security services such as disk encryption, credential protection, and multifactor authentication. It works integrated with the operating system and is supported on various platforms including Windows, Linux, and macOS.

5. Secure Startup in Operating Systems

Modern operating systems (such as Windows and Linux) are not satisfied with just firmware-level security, but have developed a robust Secure Startup Framework, combining Secure Boot and the Trusted Platform Module (TPM). The aim is to ensure a secure environment from the very beginning of system startup until the operating system kernel loads.

1. Windows Secure Startup

Secure Boot is enabled by default in Windows operating systems (UEFI required). The Secure Boot process first verifies the digital signature of each bootloader, driver, and OS kernel. Only components signed by Microsoft or the vendor can run. BitLocker Drive Encryption is then enabled using the TPM hardware. The TPM securely stores the key used for disk encryption and prevents the drive from being unlocked if the correct boot environment is not available.

TPM is also used in Windows through Measured Boot. Here, a hash value of each boot step is stored in the TPM, which is later used to verify integrity.

This ensures that Windows does not allow rootkits or boot kit malware to enter the initial boot step.

2. Linux Secure Startup

Although Secure Boot and TPM are used in the same way in Linux systems, users have more configuration freedom here.

Linux Secure Boot when enabled only loads signed kernels and modules. The signing keys from Canonical (Ubuntu), Red Hat (RHEL/Fedora), or SUSE are used depending on the distribution. TPM-enabled Linux systems use tpm-tools and IMA (Integrity Measurement Architecture) that record the hashes of each file and kernel module in the TPM.

Linux also brings in dm-crypt + LUKS along with the TPM to harden disk encryption. This ensures that only the authorized bootloaders and kernels can be started the OS.

In this way, Linux gets a Trusted Boot Environment that is versatile but very secure.

6. Real-World Use Case: Windows + Secure Boot + TPM + BitLocker

The combination of Secure Boot, Trusted Platform Module (TPM), and BitLocker Drive Encryption in the Windows operating system is a classic example of modern cybersecurity. This integrated framework provides a complete layer of security from the initial stages of system startup to user data.

1. Secure Boot: Ensuring a Trusted Startup

Secure boot is enabled in the process of starting Windows and it verifies the digital signature of each boot component, for instance, the bootloader, drivers, and the OS kernel. Thus, it is ensured that solely the software that are both allowed and have not been altered are loaded. By now, such malicious software as rootkits, boot kits, or viruses, that have not yet been identified are blocked from entering the system.

2. TPM: Hardware-Based Key Protection

Then comes the TPM. The encryption key that BitLocker uses is protected at the hardware level within the TPM. This key cannot be read by ordinary software in any way

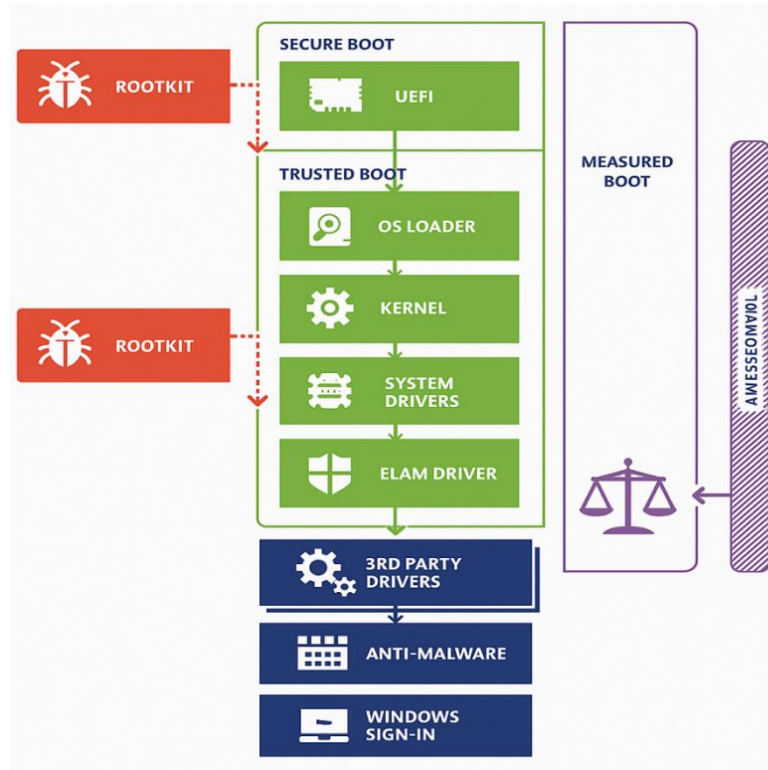


Figure-2: Secure Boot, Trusted Boot, and Measured Boot block malware at every stage

The TPM releases the key only when the correct boot configuration and valid signature are found. As a result, even if an attacker connects the hard drive to another computer, the data cannot be accessed.

3. BitLocker: Full Disk Encryption

BitLocker drive encryption technology secures the disk with a key that is stored in the TPM. In case an illegal alteration has happened to the start-up process (for instance bootloader modification or BIOS tampering), the TPM will not give the key, thus the drive will remain locked. That way data is kept safe from being accessible at any time.

4. End-to-End Chain of Trust

Secure Boot → Bootloader Verification → OS Kernel Integrity → BitLocker Encryption

This sequential security not only Guardians the entire Chain of Trust in Windows, but also the boot process barrier system.

7. Challenges and Limitations

The Secure Boot system is designed to keep out viruses and other malicious software, but there are some limitations and challenges to its functionality.

1. Dual-Boot Restrictions

Secure Boot makes it tricky to use several operating systems. As an illustration, if one has Windows and Linux installed, the Linux bootloader might not be recognizable by Secure Boot because of the missing signature, therefore, the latter is blocking it. Hence a large number of users have to disable Secure Boot while using a dual boot.

2. Firmware Updates

Secure Boot often causes compatibility issues when updating firmware (UEFI/BIOS). New firmware does not always support old keys, and if new keys provided by the vendor do not arrive in time, the system may experience problems.

3. Secure Boot Bypass Methods

Even though Secure Boot is very effective in blocking malware, the hackers with high skills have tried to get around it using vulnerabilities. In a few cases, misconfigurations, weak keys, or rootkits have been found to go through Secure Boot, which means that the security of it has been questioned, in general.

4. Usability vs. Security Trade-off

If the Secure Boot is on, the unsigned drivers, custom kernels, and experimental software are automatically blocked. This is a limitation in research or development environments, as it reduces user freedom and flexibility.

8. References (Harvard Style)

- Microsoft. (2023) *Secure Boot overview*. Available at: <https://learn.microsoft.com/en-us/windows-hardware/design/device-experiences/oem-secure-boot> (Accessed: 30 August 2025).
- Microsoft. (2023) *BitLocker overview*. Available at: <https://learn.microsoft.com/en-us/windows/security/information-protection/bitlocker/bitlocker-overview> (Accessed: 30 August 2025).
- Bulusu, M. and Zimmer, V. (2015) *UEFI Plugfest 2015 – Challenges for UEFI and the Cloud*. White Paper.
- Stallings, W. (2021) *Computer organization and architecture: Global edition*. Harlow: Pearson Education Limited.
- Microsoft. (2025) *Trusted Platform Module technology overview*. Microsoft Learn. Available at: <https://learn.microsoft.com/en-us/windows/security/hardware-security/tpm/trusted-platform-module-overview> (Accessed: 30 August 2025).
- Malwarebytes. (2025) *What is firmware?* Malwarebytes. Available at: <https://www.malwarebytes.com/cybersecurity/computer/what-is-firmware> (Accessed: 30 August 2025).